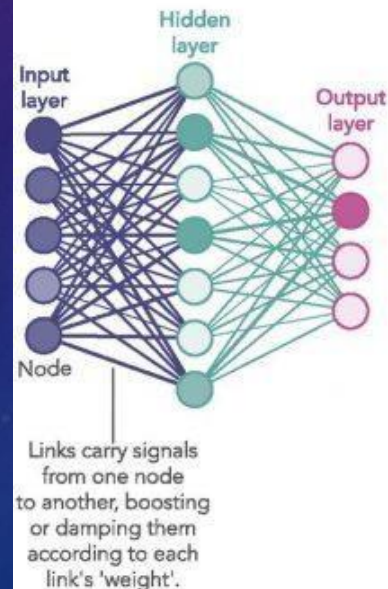


CSE4006- Deep

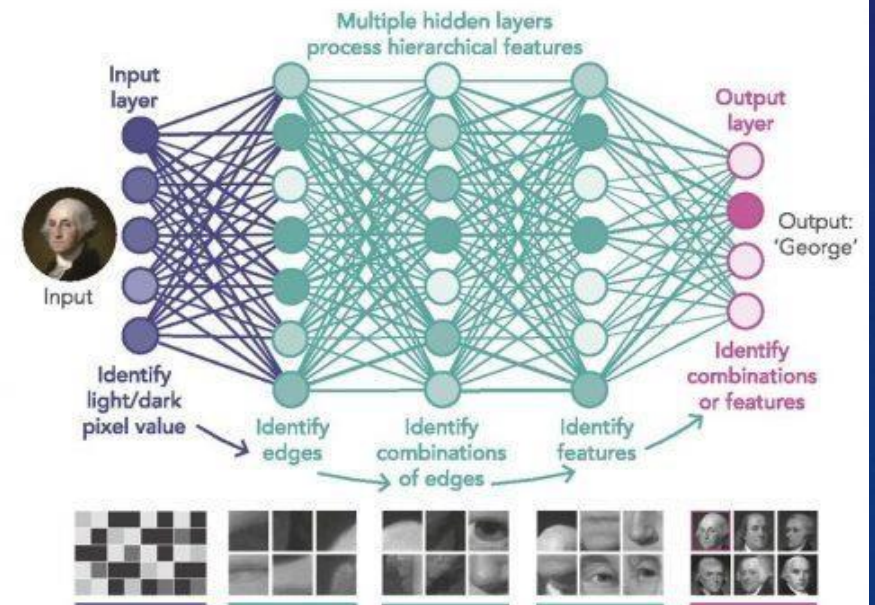
ng



1980S-ERA NEURAL NETWORK

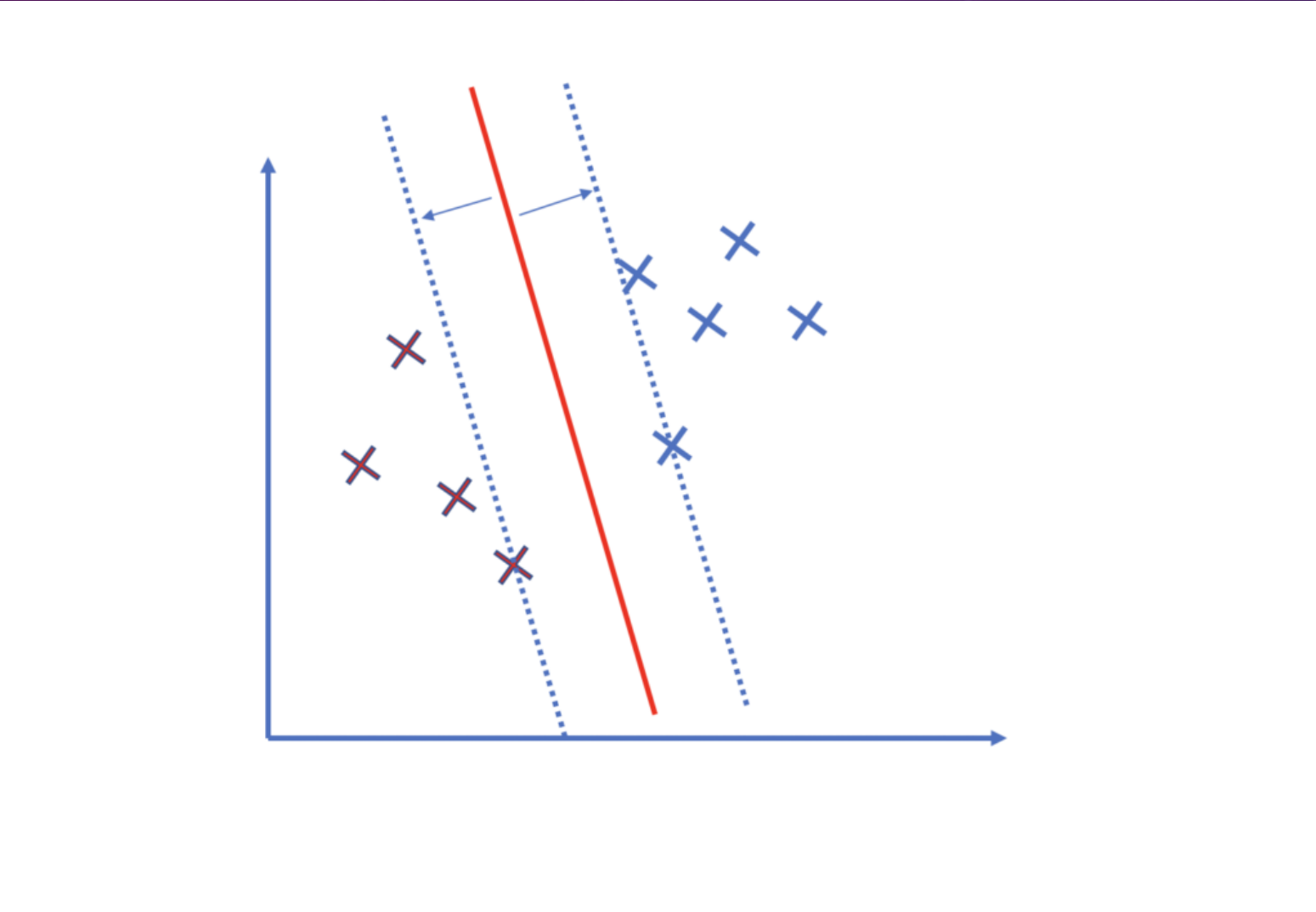


DEEP LEARNING NEURAL NETWORK



Loss functions

- A function that calculates loss for 1 data point is called the loss function.
- It's a method of evaluating how well your algorithm models your dataset.
- The **loss function** is the function that **computes the distance between the current output of the algorithm and the expected output**.
- It's a method to evaluate how your algorithm models the data.
- It can be categorized into two groups. One for **classification** (discrete values, 0,1,2...) and the other for **regression** (continuous values).
- **Regression Loss Functions**
 - Mean Squared Error Loss
 - Mean Squared Logarithmic Error Loss
 - Mean Absolute Error Loss
- **Binary Classification Loss Functions**
 - Binary Cross-Entropy
 - Hinge Loss
 - Squared Hinge Loss
- **Multi-Class Classification Loss Functions**
 - Multi-Class Cross-Entropy Loss
 - Sparse Multiclass Cross-Entropy Loss
 - Kullback Leibler Divergence Loss



REGRESSION LOSS FUNCTIONS

Mean Squared Error

- Mean square error is calculated by taking the average, specifically the mean, of errors squared from data as it relates to a function.
- A smaller MSE is preferred because it indicates that your data points are dispersed closely around its central moment (mean).
- Sensitive to outliers, punishes larger error more
- Lesser the MSE => Smaller is the error => Better the estimator.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

Mean Absolute Error

- MAE takes the average sum of the absolute differences between the actual and the predicted values.
- Treat larger and small errors equally. Not sensitive to outliers
- The lower the MAE, the higher the accuracy of a model.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

HINGE LOSS

- Primarily developed for use with Support Vector Machine (SVMs) models.
- if a computed value predicts a correct classification and is larger than the margin then *there will be no hinge loss*.
- If a computed value gives a correct classification but is too close to zero (where too close is defined by a margin) there is a small hinge loss.
- And, if a computed value gives an incorrect classification there will always be a hinge loss with quite high value.

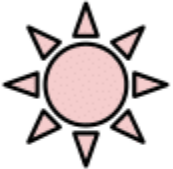
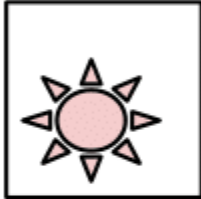
margin = 0.2

	actual	computed	hinge loss
=====			
[0]	+1	0.55	0
[1]	+1	0.25	0
[2]	+1	0.15	0.07
[3]	+1	-0.25	0.49
[4]	-1	-0.35	0
[5]	-1	-0.98	0
[6]	-1	-0.05	0.16
[7]	-1	+0.25	0.49

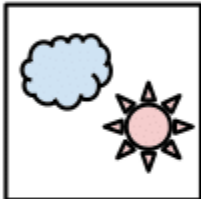
$Loss = \max(0, 1 - y \cdot \hat{y})$
where, y is the actual value
 $\hat{y}$ is the predicted value.

TASKS

Multi-Class

C = 3	Samples
  	  
	Labels (t)
	$[0\ 0\ 1]$ $[1\ 0\ 0]$ $[0\ 1\ 0]$

Multi-Label

Samples
  
Labels (t)
$[1\ 0\ 1]$ $[0\ 1\ 0]$ $[1\ 1\ 1]$

CROSS ENTROPY AND BINARY CROSS ENTROPY

- ***Cross-entropy*** builds upon the idea of information theory entropy and measures the difference between two probability distributions for a given random variable/set of events.
- Cross entropy can be applied in both binary and multi-class classification problems.
- **Binary cross-entropy** is a loss function that is used in binary classification problems. The main aim of these tasks is to answer a question with only two choices.

BINARY CROSS ENTROPY

$$BCE = -\frac{1}{n} \sum_{i=1}^n (y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i))$$

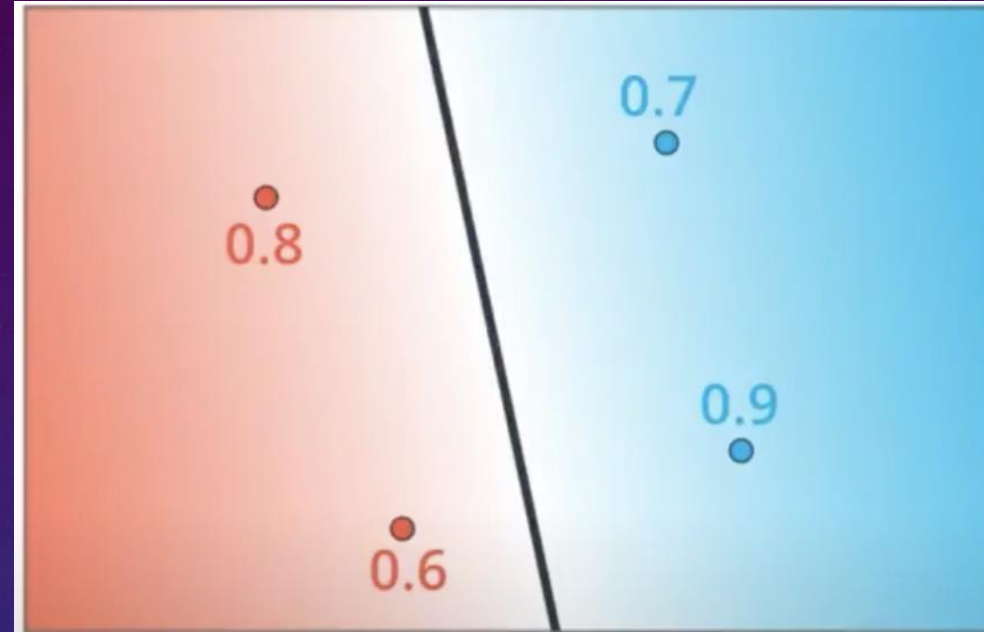
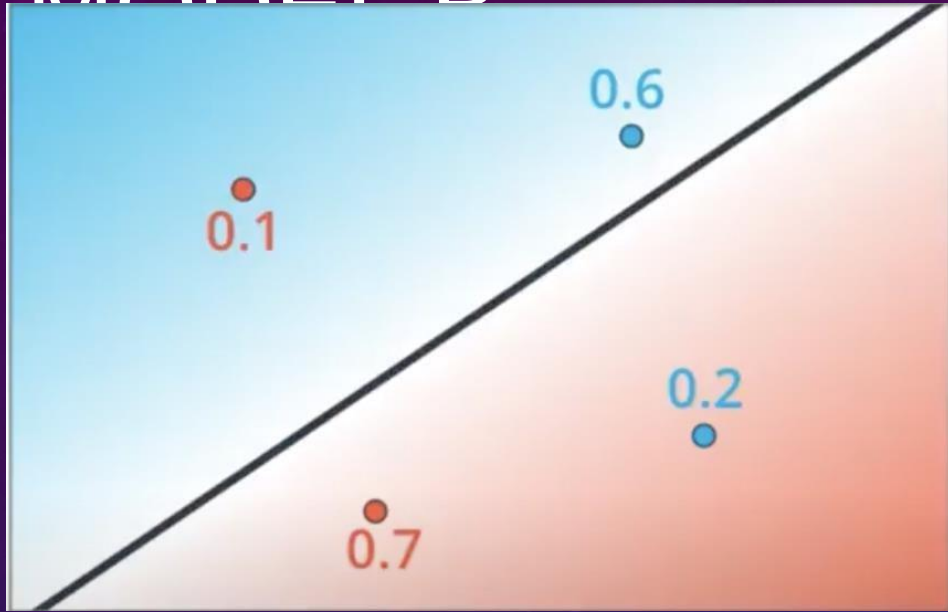
- n - the number of data points.
- y - the actual label of the data point. Also known as true label. Can only be 0 or 1.
- $\hat{y}$ - the predicted probability of the data point. Between 0 and 1. This value is returned by model.

actual labels of y	1	0	0	1
predicted probabilities of $\hat{y}$	0.8	0.2	0.6	0.9

Here is example how BCE can be calculated using these numbers:

$$\begin{aligned}
 BCE &= -\frac{(1 \cdot \log(0.8) + (1 - 1) \cdot \log(1 - 0.8)) + (0 \cdot \log(0.2) + (1 - 0) \cdot \log(1 - 0.2)) +}{4} = \\
 &= \frac{+(0 \cdot \log(0.6) + (1 - 0) \cdot \log(1 - 0.6)) + (1 \cdot \log(0.9) + (1 - 1) \cdot \log(1 - 0.9))}{4} \approx \\
 &\approx -\frac{-0.223 - 0.223 - 0.916 - 0.105}{4} \approx 0.367
 \end{aligned}$$

MODEL A AND MODEL B



- **Product probability:** *The probability of two(or more) independent events that are occurring together is calculated by multiplying the events' individual probabilities.*
- **We want to calculate the total probability of the models by multiplying the probability of each independent student.**
- **Product Probability Model A:**
 - $0.1 * 0.7 * 0.6 * 0.2 = 0.0084$
- **Product Probability Model B:**
 - $0.8 * 0.6 * 0.7 * 0.9 = 0.3024$
- The product probability for model B is better than that of A.

- *Product probability* works better when we have a few items to predict, but this is not the case with real-life model predictions.
- For instance, if we have a class full of 1000 students, the product probabilities will always be closer to 0, regardless of how good your model is. If we also change one probability, the product will change drastically and give the wrong impression that a model performs well. So, we need to transform the products to a sum using a **logarithmic function**.
- Log Model A:
 - $\log(0.1) + \log(0.7) + \log(0.6) + \log(0.2)$
 - $-1 + -0.154 + -0.221 + -0.698 = -2.073$
- Log Model B:
 - $\log(0.8) + \log(0.6) + \log(0.7) + \log(0.9)$
 - $-0.09 + -0.22 + -0.15 + -0.045 = -0.505$

- The log of a number between 0 and 1 will always be negative. Is the above a better way to evaluate our model performance? Not really. Instead, we'll take the **negative logarithm of predicted probabilities**.
- Negative Logs Model A:
 - $-\log(0.1) + -\log(0.7) + -\log(0.6) + -\log(0.2)$
 - $1 + 0.154 + 0.221 + 0.698 = 2.073$
- Negative Logs Model B:
 - $-\log(0.8) + -\log(0.6) + -\log(0.7) + -\log(0.9)$
 - $0.09 + 0.22 + 0.15 + 0.045 = 0.505$
- Cross-entropy loss is *the sum of the negative logarithm of predicted probabilities of each student*.
- *Model A's cross-entropy loss is 2.073; model B's is 0.505. Cross-Entropy gives a good measure of how effective each model is.*

`1`
`= 'sigmoid'`
`compile(loss='binary_crossentropy',`
`metrics=['accuracy'])`

	A	B	C	D
Probabilities	P1=0.8(fail)	P2=0.6(fail)	P3=0.7(pass)	P4=0.9(pass)
Pass probabilities	1 – P1	1 – P2	P3	P4
y _i = 1 if student passes else 0, therefore:				
	y1 = 0	y2 = 0	y3 = 1	y4 = 1

BINARY CROSS ENTROPY- FORMULA

$$\text{Loss} = -\frac{1}{\text{output size}} \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i + (1 - y_i) \cdot \log (1 - \hat{y}_i)$$

output size is the number of scalar values in the model output.

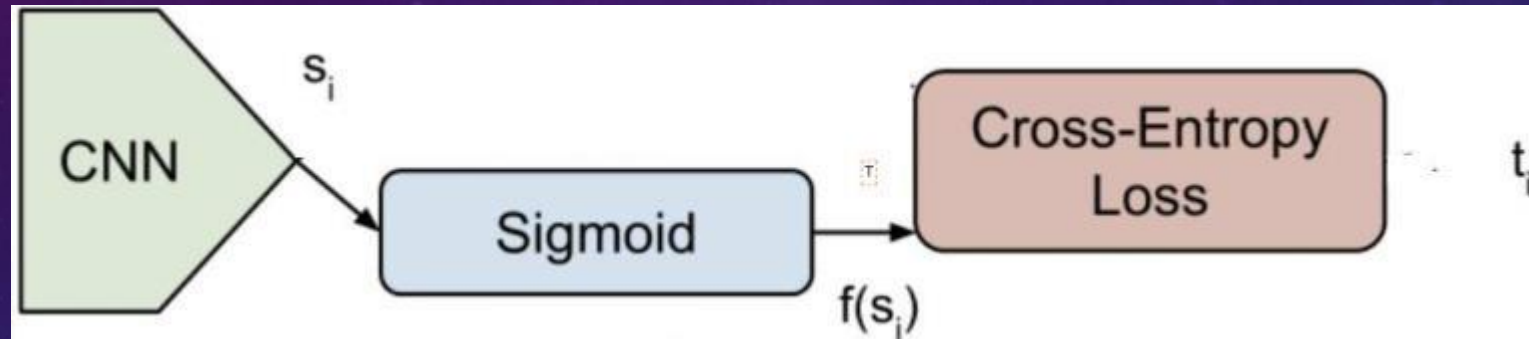
- **Cross entropy for student C:**
$$-\sum_{n=i}^c y_i \log(p_3) + (1 - y_i) \log(1 - p_3)$$

- Blue represents a student pass. Red represents student failure.

$$-\sum_{n=i}^c (1 - y_i) \log(1 - p_1) + y_i \log(p_1)$$

- `Model.add(dense=1,activation='sigmoid')`
- `Model.compile(loss='binary_crossentropy',optimizer=opt,metrics=['accuracy'])`

- we're calculating cross entropy using predicted probabilities of each student.
- We'll incorporate the formula to include how the probabilities are generated.
- sigmoid activation function could be used in binary classification to transform linear function scores into probabilities.
- Here's the cross-entropy function using the activation:



- S_i – inputs/weights
- f – the activation function in this case,
- t – the target predictions
- i – the class to predict.

BCE=

$$-\sum_{n=i}^c t_i \log(f(s_i)) + (1 - t_i) \log(1 - f(s_i))$$

MULTI-CLASS CROSS-ENTROPY / CATEGORICAL CROSS-ENTROPY

- The probabilities of each container need to sum to 1.
- If you are dealing with a multi-class classification problem you can calculate the Log loss in the same way

$$L = -\frac{1}{m} \sum_{i=1}^m y_i \cdot \log(\hat{y}_i)$$

Fruit	Container A probabilities	Container B probabilities	Container C probabilities
Orange	0.7	0.3	0.1
Apple	0.2	0.4	0.5
Lemon	0.1	0.3	0.4

	Container A	Container B	Container C
Correct fruits in the Respective containers	Oranges	Lemon	Lemon
The predicted probability that the fruit is correct	0.7	0.3	0.4

- Product probabilities = $0.7 * 0.3 * 0.4 = 0.084$
- Cross Entropy = $-\log(0.7) + -\log(0.3) + -\log(0.4) = 1.073$